

llm-proxy

Защита персональных данных
при работе с LLM

Обнаружить. Заменить. Восстановить.

Команда «Красный Kod»

Полезный запрос не должен раскрывать данные

Приложение сохраняет смысл задачи, передавая модели обозначения вместо персональных данных.

ИСХОДНЫЙ ТЕКСТ

Напишите на
ivan@example.com

ТЕКСТ ДЛЯ LLM

Напишите на
<EMAIL_1>

Адрес нужен приложению. Для составления ответа модели достаточно обозначения.

Один API — два направления обработки

POST /process принимает текст и payload_id, возвращает обработанный result.

01

Обнаружить

Правила распознавания
и контекст фразы

02

Заменить

Уникальные обозначения
и временное состояние

03

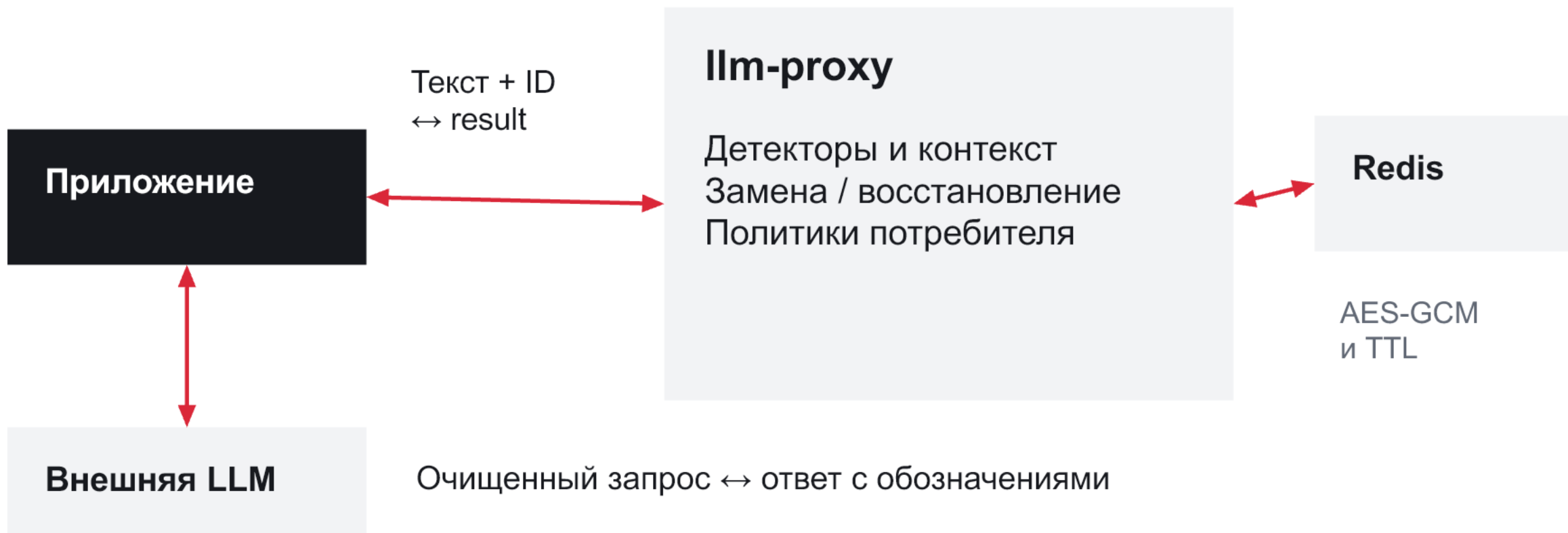
Восстановить

Известные обозначения
в новом ответе модели

Повтор исходного запроса с тем же ID возвращает ту же маску.

Приложение управляет всем маршрутом

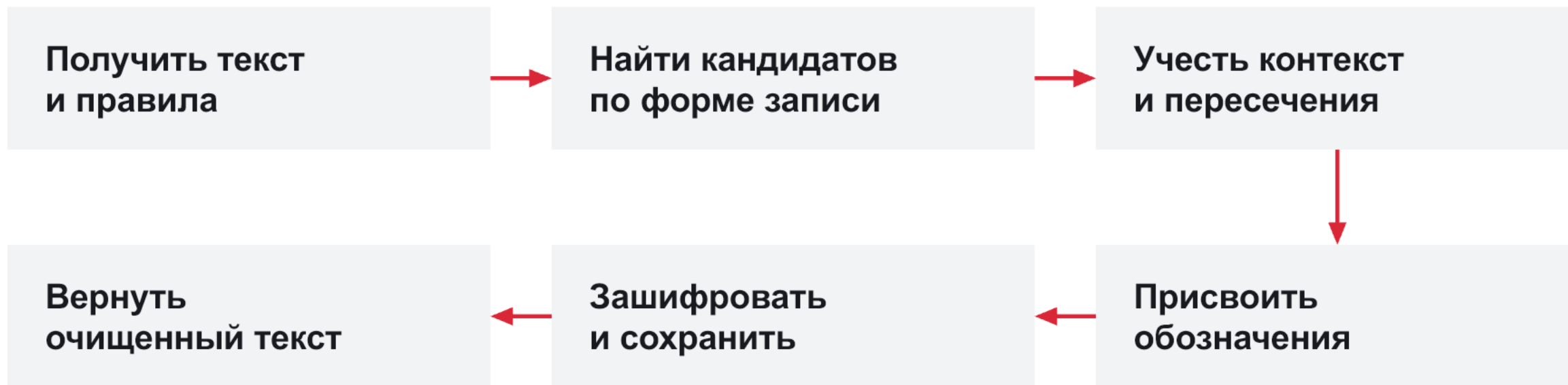
llm-proxy обрабатывает данные; приложение самостоятельно вызывает внешнюю LLM.



Python · FastAPI · Redis · Docker

От текста к безопасным обозначениям

Локальные правила и проверка контекста. Нейросеть для распознавания не вызывается.



ivan@example.com → **<EMAIL_1>**

Восстанавливаем данные в новом ответе LLM

Тот же `payload_id` связывает ответ модели с сохранёнными соответствиями.

01 ИСХОДНЫЙ ЗАПРОС

Напишите на `ivan@example.com`

02 ПОСЛЕ МАСКИРОВАНИЯ

Напишите на `<EMAIL_1>`

03 НОВЫЙ ОТВЕТ LLM

Сегодня рекомендую отправить сообщение на `<EMAIL_1>`

04 ПОСЛЕ ВОССТАНОВЛЕНИЯ

Сегодня рекомендую отправить сообщение на `ivan@example.com`

Порядок и повторы масок могут меняться. Остальной текст ответа сохраняется.

Персональные данные — по категориям

Структурные проверки дополняются контекстом: дата или число сами по себе ещё не ПД.

Личные сведения

ФИО, дата и место рождения, гражданство

Контакты и адреса

Телефон, электронная почта, личный адрес и его части

Документы

Паспорт и его выдача, ИНН, водительское удостоверение

Финансовые данные

Номер карты, держатель, защитный код и ПИН-код

Контекст уменьшает лишние замены; распознавание произвольного текста не гарантировано.

Чувствительное состояние живёт недолго

Исходный и замаскированный текст, соответствия и правила шифруются перед записью.

15 минут

после создания записи

2 минуты

после успешного восстановления

AES-GCM · Redis в частной сети

Правила потребителя задаются конфигурацией

В логах и метриках нет исходных ПД

Публичное демо без отдельной авторизации /process: только синтетические данные.

Локально: 1 000 запросов в секунду

Полный профиль — 300 секунд, 8 рабочих процессов. Это результат локального стенда.

300 001

успешный запрос

21,78 мс

задержка p95

0

ошибок и 429

F1 = 1,0

46 синтетических примеров
F1 объединяет точность и полноту обнаружения.

Небольшой локальный корпус не заменяет скрытую проверку организаторов.

Railway: демо работает, 1 000 RPS не пройдены

Результаты внешних прогонов 23 сентября 2026 года.

100 ЗАПРОСОВ/С · 30 СЕКУНД

PASS

3 001 запрос · 0 ошибок

p95 = 86,04 мс

Без 429 и пропущенных запусков

1 000 ЗАПРОСОВ/С · 300 СЕКУНД

BLOCK

≈645 успешных запросов/с

≈33% ошибок; в том числе ≈76 тыс. 429

p95 ≈10 с · ≈10 тыс. dropped

Локальный результат 1 000 RPS нельзя приписывать облачному развёртыванию.

Демо и подключение через POST /process

Приложение отправляет очищенный текст в LLM и возвращает её ответ с тем же payload_id.

ПРИМЕР ЗАПРОСА

```
{  
  "payload": "Напишите на ivan@example.com",  
  "payload_id": "demo-001"  
}
```

ОТВЕТ

```
{ "result":  
  "Напишите на <EMAIL_1>" }
```

Демо / Swagger: llm-proxy-production-84c7.up.railway.app/docs

Код: github.com/rp-alpha-vibe/llm-proxy · доступ к приватному репозиторию — отдельно

Защита данных без потери смысла ответа

- 01 Распознаём ПД локальными правилами и контекстом
- 02 Заменяем данные и восстанавливаем их в новом ответе LLM
- 03 Храним зашифрованное состояние с ограниченным сроком
- 04 Предоставляем работающий HTTP API и облачное демо

Ограничение: полный профиль Railway не пройден; официальная оценка ещё не подтверждена.